

How the Python course works

Thirteen lessons of Python, written and run in your browser. Everything you need to know before you open lesson 1 is on these pages. Keep it beside you.

Before you start

Python is a language for writing instructions. A computer follows them one line at a time, from the top. A set of instructions is a **program**, and everything in this course is you writing, reading or fixing one.

You do not install anything. Python runs inside the browser, so what you write runs on the page in front of you and nobody else can see it.

You cannot break anything. If your program goes wrong, the worst that happens is a message in red. If it never stops, the page stops it for you after a few seconds. Nothing you type can damage the computer, the website or your work.

The screen, part by part

1 Printing output

1 BUILD IT · ACTIVITY 5 OF 10

2 YOUR TASK

1. Display the message Hello world.

• Nothing is typed in by the user.

3 LEARN

The quotation marks say where your text starts and stops.

4 print("Hello world")

5 ▶ Run

Syntax reminder

6 ONE RUN OF YOUR PROGRAM

You type: nothing · It displays: Hello world

7 PROGRAM OUTPUT

Hello world

8 Hint

9 Check my answer

- What kind of activity this is**, and where you are in the station.
- Your task**, one numbered step at a time. Read every step before you start.
- Learn** — a worked example on different values, showing the shape of what you need.
- The editor**, where you write your program. Tab indents; Escape leaves the editor.
- Run** does what your instructions say. It never marks anything.

- One run of your program** — what gets typed in, and what must come out.
- Program output** — everything your program displayed, and any error.
- Hint** gives you one step at a time. None of them is the answer.
- Check my answer** marks your program. You may check as often as you like.

Revise 360 - Python course guide

Page 1 of 6

Run and Check are not the same button

► Run

Does exactly what your instructions say and shows the result under **Program output**. It never marks anything and never tells you that you are wrong.

Press it as often as you like. Trying something out is how programming is done.

Check my answer

Runs your program against the tests for that activity and marks it. You will see which tests passed, what was expected, and what your program displayed instead.

You may check as many times as you like. **Your best mark is the one that is kept**, so a program you get working on the fourth try scores the same as one that works first time.

The seven kinds of activity

A technique is never taught all at once. You run it, read it, change it, complete it, fix it, and only then write it from nothing. The label at the top left of every activity tells you which of those you are doing.

SHOWN AS	WHAT YOU DO	HOW IT IS FINISHED
Try it	Run a program that already works and watch what it does.	Nothing is marked. Pressing Run finishes it.
Predict	Read a program and say what it will display, before running it.	Choose one of three answers. You are then shown what Python really does.
Change it	A working program, with one thing to alter.	Check my answer runs your program against the tests.
Complete it	Part of a program is missing. Fill in the gap.	Check my answer runs your program against the tests.
Fix it	A broken program. Find the mistake and put it right.	Check my answer runs your program against the tests.
Build it	Write the program yourself.	Check my answer runs your program against the tests.
Challenge	The hardest activity on a station, at the end of it.	Check my answer runs your program against the tests. It is not optional.

Across the whole course there are 614 activities: 79 to run, 109 to predict, 56 to change, 74 to complete, 76 to fix and 220 to write, of which 78 are the challenge at the end of a station. You complete all of them, in order.

Getting unstuck

The Hint button

A hint is a ladder, not an answer. Press it once for the first step, and again for the next when you need it. Most activities have three or four steps:

1. **Think** — a sentence naming the idea you need.
2. **The Python you need** — the shape of it, with this activity's own values left out.
3. **How it starts** — the first line or two.
4. **Watch the technique** — a short animation of how it works, on different values.

None of the steps is the answer to the activity, so there is nothing to be gained by skipping to the last one. Your program stays exactly where you left it while the hint is open.

The Syntax reminder button

Every command the course has taught you so far, with what it does and an example. It does not mention the activity you are on. Look things up in it as often as you like — nobody remembers whether the brackets go on `print` or `input` at first, and there is no advantage in guessing. The same list is printed in the **Python syntax guide** beside this one.

When something goes wrong

Errors are normal. Every programmer sees them all day. An error is not you failing; it is Python telling you it could not follow an instruction, and which one.

An error message comes in three parts. Read them in this order:

1. **Python's own words**, such as `NameError: name 'score' is not defined`. Learning to read these is part of learning to program.
2. **What this probably means** — in plain English. *Probably*, because the message names what went wrong, not always why.
3. **Check these things** — a short list of what to look at first.

These are the ones you will meet most:

PYTHON SAYS	WHAT IT USUALLY MEANS, AND WHAT TO CHECK
<code>IndentationError</code>	Python uses the spaces at the start of a line to see what is inside a loop or an if. • Is every line inside the block indented by the same amount? • Did you mix tabs and spaces? • Does the line after a <code>:</code> go in four spaces?
<code>NameError</code>	Nothing called that name exists yet. • Is that name spelled the same everywhere? • Did you give that name a value before using it? • If that name is text, does it need quotation marks around it?
<code>TypeError</code>	You tried to join text and a number with <code>+</code> . • Put the number inside <code>str()</code> to turn it into text. • Or print the parts separately, one in each <code>print()</code>.
<code>ValueError</code>	<code>int()</code> was given something that is not a whole number. • Does the value have a decimal point in it? <code>float()</code> handles those. • Is there a space or a letter in the value?
<code>IndexError</code>	You asked for a position that is not there. • The first item is at 0, so the last one is at <code>len() - 1</code>. • Is the list as long as you think? <code>print()</code> it and look.
<code>ZeroDivisionError</code>	Something was divided by zero. • Could the value you divided by be 0? • Check the count before dividing by it.
<code>SyntaxError</code>	Python could not make sense of that line. • Is a <code>:</code> missing at the end of an if, for, while or def? • Are the quotation marks in a pair? • Is a bracket missing or spare?
<code>EOFError</code>	The program asked for more typed-in values than it was given. • Count the <code>input()</code> lines: is one more than the task needs? • Is an <code>input()</code> inside a loop that runs too many times?

If your program never stops, you have written a loop with no way out. It is the commonest mistake in the course and everybody makes it. The page stops the program for you; go back to the loop and find the line that was meant to change the value it is testing.

Working in order, and getting help

You do every question, in order. The next one opens when this one is finished, and there is no way to skip past it. That is on purpose: each activity teaches the thing the next one needs, so going round it would only make the next one harder.

- **Finished means right.** A program is finished when every test passes; a Predict when you choose the right answer; a Try it when it runs. Part of the way there is recorded, but it does not open the next question.
- **Try as many times as you like.** Your best attempt is the mark that is kept, so getting it working on the fourth try scores the same as first time.
- **Asking for help costs you nothing.** It takes no marks off and it is not a last resort.
- **Hints do not finish a question for you,** and neither does running out of attempts. Only doing the work does.
- Your work saves after every answer. You can stop, log out and come back to the same question.
- **Review mode** lets you look back over anything you have already finished.

When you are stuck

Press **I need help**. It is there from the first attempt, on every question. It will offer you a hint, take you back to the worked example, or tell you to ask your teacher.

Not sure what to do next? That is OK. You can use a hint or ask your teacher for help. Show them this question and your code. Complete this question before moving on.

Your teacher cannot see your screen from their desk, so put your hand up or message them the way your school normally does. If you think the question itself is wrong, tell them — reporting a fault does not finish the question, and it should not.

How to work through an activity

1. Read **every** numbered step, and the bullets under them, before you type anything.
2. Look at **One run of your program**. It tells you exactly what gets typed in and exactly what must come out. Most wrong answers are right programs that display the wrong thing.
3. Read the **Learn** example. It shows the shape on different values, never the answer.
4. Write a little, then press **Run**. Do not write the whole thing and hope.
5. Press **Check my answer**. If a test fails, read what it expected and what yours displayed. The difference is the bug.
6. Change **one thing**, then check again.

On paper first. For anything longer than three lines, write down what goes in, what has to be worked out, and what comes out, before you touch the keyboard. Your worksheet has space for exactly that.

The thirteen lessons

#	LESSON AND ITS SIX STATIONS	ACTIVITIES
01	Printing and variables Printing output · Variables · Reading input · Numbers and text · Casting · Putting it together	55
02	Arithmetic The four operations · Whole number division · Powers and roots · Rounding · Random numbers · Putting it together	53
03	Selection Comparing values · if and else · elif · AND, OR and NOT · Decisions inside decisions · Putting it together	54
04	Count-controlled loops Repeating a line · Using the counter · Choosing the range · Running totals · Loops inside loops · Putting it together	54

05	Condition-controlled loops	54
	A loop with a condition · Asking until it is right · Sentinel values · Counting inside a while · Choosing for or while · Putting it together	
06	String handling	43
	Length and position · Taking part of a string · Upper and lower case · Searching inside text · Character codes · Putting it together	
07	Lists	43
	Making a list · Reading an item · Going through a list · Adding and removing · Totals and averages · Putting it together	
08	Two-dimensional lists	42
	A list of lists · Rows and columns · Going through a grid · Totalling a row · Totalling a column · Putting it together	
09	Subprograms	42
	Writing a procedure · Parameters · Returning a value · Functions and procedures · Local and global · Putting it together	
10	File handling	42
	Reading a whole file · Reading line by line · Writing to a file · Adding to a file · Records in a file · Putting it together	
11	Searching and sorting	44
	Linear search · Binary search · Bubble sort · Insertion sort · Merge sort · Putting it together	
12	Validation and authentication	45
	Checking the input · Range checks · Length and format checks · Keeping asking · Authentication · Putting it together	
13	SQL queries	43
	SELECT and FROM · WHERE · Comparing values · AND and OR · Sorting results · Putting it together	

This course sits beyond the OCR J277 specification. It teaches the same programming techniques the specification lists, as code you write and run.

Words this course uses

Program	A list of instructions a computer follows, in order, from the top.
Statement	One instruction. Usually one line.
Output	Anything your program displays on the screen.
Input	Anything the person using the program types in.
Variable	A name your program remembers a value under.
Value	The thing a variable is holding at the moment.
String	Text. It goes inside quotation marks.
Integer	A whole number, with no decimal point.
Float	A number with a decimal point in it.
Boolean	A value that is either True or False.
Casting	Changing a value from one type into another, with <code>int()</code> , <code>float()</code> or <code>str()</code> .
Concatenation	Joining two strings end to end with <code>+</code> .
Condition	A question with a True or False answer, like <code>score > 10</code> .
Selection	Choosing what happens next, using <code>if</code> , <code>elif</code> and <code>else</code> .
Iteration	Repeating. A <code>for</code> loop repeats a set number of times; a <code>while</code> loop repeats while a condition is true.
Accumulator	A variable that keeps a running total.
List	A set of values kept under one name, numbered from 0.
Index	The position of one item. The first is 0.
Subprogram	A named piece of program you can use again. A function hands a value back; a procedure does not.
Parameter	A value handed to a subprogram when it is called.
Validation	Checking that what was typed in is sensible before using it.
Syntax error	Python could not read the line at all.
Logic error	The program runs, but does the wrong thing.

Revise 360 — a product of Revise 360 Ltd, created by Olly Pettitt. An independent resource: not written, endorsed or approved by OCR or any exam board. Classroom use and personal study only.